COS 568 Project: Milestone 1 Report

Setup

The experiment in Milestone 1 was done on Princeton's Adroit Cluster, as recommended by the project specification. I used the login node to compile executables using the build scripts and employed the compute nodes (via *sbatch*) to run them on*/scratch/network/hj7206/*. Following the project spec, the Python environment was loaded using *module load anaconda3/2023.3.* All experiments were repeated three times using the *-r 3* flag, producing 3 throughput values per configuration. Below, we report the arithmetic mean of these three runs, selecting the configuration with the highest average throughput as written in the spec. The three datasets used are *books, fb, and osmc*.

Results

Note to Course Staff: I did not use provided analysis.py as is, but edited it below to better visually see the differences in throughput performance across indexes within a dataset. I hope this is fine since the conclusion is the same regardless (just aesthetic changes).

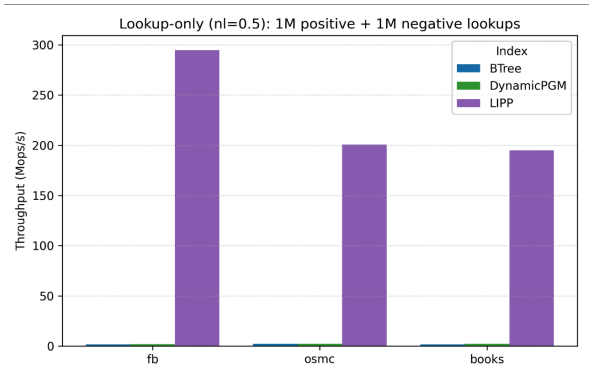

Figure 1: Lookup-only workload with 1M positive lookups and 1M negative lookups (total 2M operations) across three datasets (fb, osmc, books) using three indexes (BTree, DynamicPGM, and LIPP). Notably, BTree and DynamicPGM have comparatively much lower throughput (Mops/s) compared to LIPP, which dominates the throughput performance..

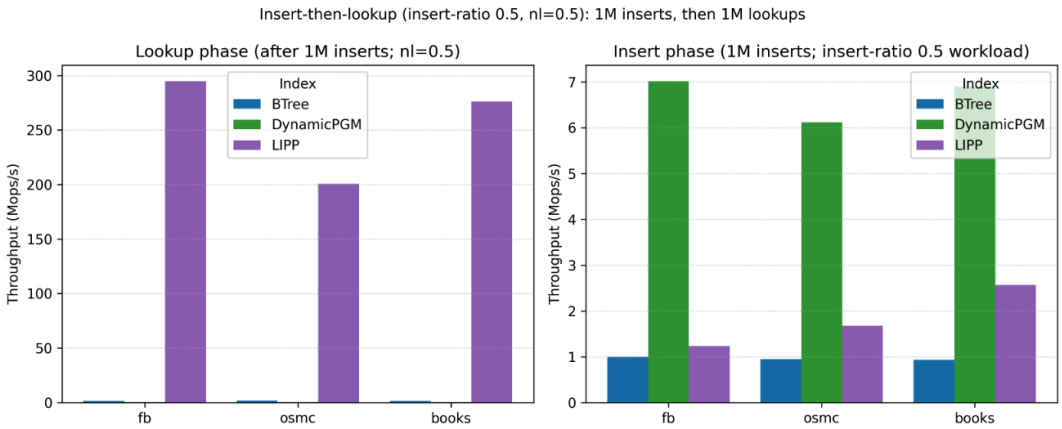

Figure 2: Insert-Lookup workload with 1M insertions, 0.5M positive lookups, and 0.5M negative lookups (total 2M operations) across three datasets (fb, osmc, books) using three indexes (BTree, DynamicPGM, and LIPP). All 1M insertion operations occur first (see right graph), followed by the 1M lookup operations (see left graph), which shows that indexes have different throughputs depending on the lookup vs. insert phase. In the insert phase, DynamicPGM dominates the throughput performance with roughly 6-7 Mops/s across the datasets, followed by LIPP and then BTree. As a result, although DynamicPGM is best for inserts, LIPP's much higher lookup throughput dominates the overall workload performance, making it the strongest index overall for this insert-then-lookup setting.

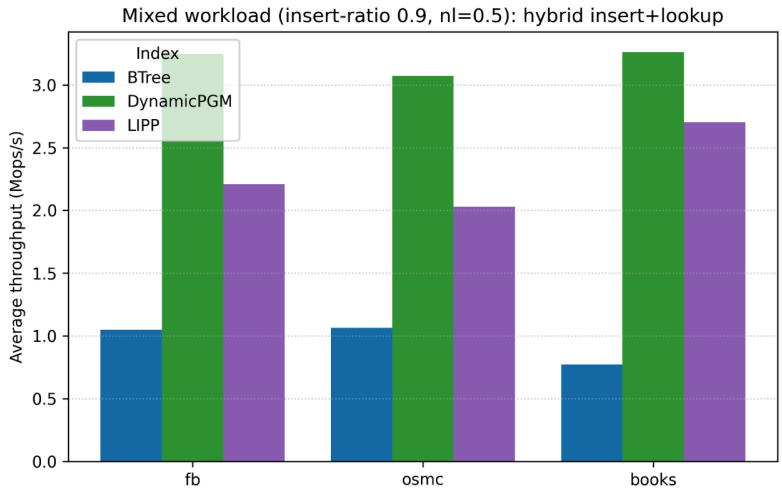

Figure 3: Mixed Insert-Lookup workload with 1.8M insertions, 0.1M positive lookups, and 0.1M negative lookups (total 2M operations) across three datasets (fb, osmc, books) using three indexes (BTree, DynamicPGM, and LIPP). The insertion and lookup operations are mixed, showing the average throughput across the hybrid workloads. In the hybrid workload where insertion operations dominate, DynamicPGM has the best the throughput performance with roughly ~3Mops/s across the datasets, followed by LIPP (~2-2.5Mops/s) and then BTree (~0.75-1 Mops/s).

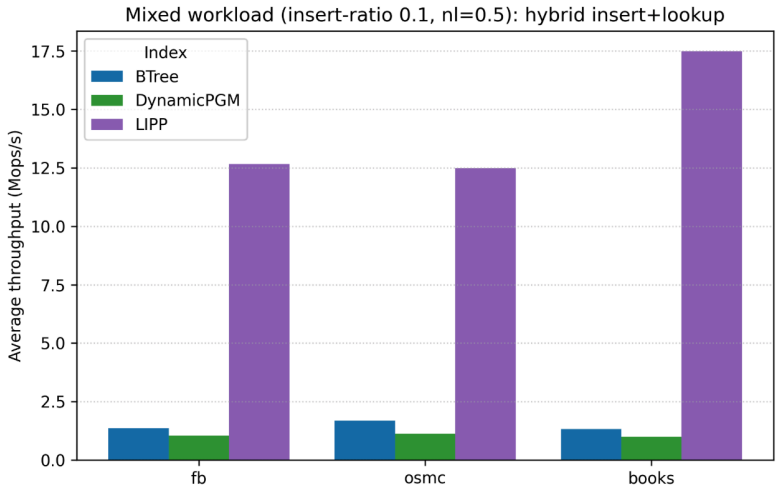

Figure 4: Mixed Insert-Lookup workload with 0.2M insertions, 0.9M positive lookups, and 0.9M negative lookups (total 2M operations) across three datasets (fb, osmc, books) using three indexes (BTree, DynamicPGM, and LIPP). The insertion and lookup operations are mixed, showing the average throughput across the hybrid workloads. In the hybrid workload where lookup operations dominate, LIPP has the best throughput performance with roughly ~12.5-17.5Mops/s across the datasets, followed by BTree and DynamicPGM which have much worse performances of (~1-2Mops/s).

Lookup Throughput: As shown in Figures 1 and 2, LIPP achieves much higher average throughput performance than the other indexing methods, with a performance of 195-295 Mops/s across the datasets. Meanwhile, DynamicPGM achieves an average of ~2-2.2 Mops/s, and BTree achieves ~1.5-2 Mops/s. This indicates that LIPP is approximately 145X better in terms of average throughput performance than the other two index methods. This trend is consistent across all three datasets, suggesting that the LIPP is the best index among the three for average lookup throughput.

Insertion Throughput: As shown in Figure 2, DynamicPGM performs the best in average throughput performance than the other indexing methods, achieving ~6-7 Mops/s, with LIPP trailing far behind second at ~1.2-2.6 Mops/s and last by BTree at ~0.9-1 Mops/s. This trend also remains consistent across the datasets. This indicates that DynamicPGM is the best for insertion workloads, performing faster than LIPP by roughly 3-4x and than BTree by 7x.

Analysis

Based on the results, we observe the differences due to fundamental data structure differences and the way each index handles data.

First, **LIPP dominates the average throughput performance for lookup operations compared to other indexes.** We observe this difference because LIPP uses model-predicted slots with very little or no search, especially when the model is precisely accurate. With precise model predictions, lookups can be very quick with high throughput and no extra in-node searching. In runtime performance, a successful lookup is O(1). Meanwhile, BTree's lookup is O(log N) runtime complexity, as one needs to traverse through the nodes in the tree to find the particular value with no shortcuts (compared to the other methods). This results in a much slower lookup time for BTree compared to LIPP, which has a constant O(1) runtime complexity. Comparatively, DynamicPGM approximates key positions using piecewise linear models and conducts a small local search within an error window – this kind of static lookup can be much

quicker than BTree (which needs to traverse the tree), but slower than LIPP due to the additional local search within the error window. LIPP, with a constant time slot lookup, explains why the index method performs nearly 145X magnitudes better than BTree and DynamicPGM.

Second, **DynamicPGM dominates the throughput performance for insertion operations compared to BTree and LIPP**. DynamicPGM was designed with a "*logarithmic* method" that maintains a hierarchy of buffers, which merge to the next level as one level fills up. In most cases, when new keys are inserted, the DynamicPGM can simply update in *constant* time in the sorted array (or simply propagate updates upwards in the data structure) at a constant cost per level. Meanwhile, for LIPP, maintaining exact slot predictions under insertions can be difficult to maintain. When insertion conflicts occur, LIPP will create new child nodes and split, which adds costs of restructuring portions of the tree and locally rebuilding the section of the tree. This increases the cost of insertion compared to DynamicPGM. For BTree, insertions require another traversal to the correct child node and potentially trigger a cascading node split up the tree, which can be extremely expensive due to pointer-heavy operations. Thus, the cheap, amortized insertions afforded by DynamicPGM's buffer hierarchy explain the higher insert throughput than BTree and LIPP, which suffer from node splits, tree restructuring, and pointer chasing operations.

Third, **BTree is conservative on both and sits at the low end for lookup and insert.** BTree uses a classic tree navigation, traversing down nodes until it finds the correct keys. Compared to LIPP (using learned model to jump to or close to the answer) and DynamicPGM (piecewise linear approximation & local search), BTree does not have any shortcuts but merely traverses the tree, affording O(log N) runtime for lookups. Meanwhile, for inserts, DynamicPGM's amortized efficient, localized updates and LIPP's local tree construction often lead to lower insertion overhead than B-trees. In contrast, B-trees may incur higher costs in the worst case due to cascading node splits that propagate up the tree.